Tecnologías IoT para la Industria 4.0 · Bloque 0

Práctica D3 · Un entorno con Compose

Describir el entorno de la práctica D2 en un compose.yaml y manejarlo con Docker Compose

Alumno	Manu Martín
Curso	2.º DAM
Fecha	31 de octubre de 2026
Objetivo	Describir en un fichero el entorno de la práctica D2 y manejarlo con Compose
Criterios	CE6.1
Entrega	`compose.yaml`, `.env.example` y tabla de decisiones (carpeta `03-compose`)
Entorno	Windows 11, Docker Desktop (motor 29.8.2, Compose 5.6.0), PowerShell

Índice

1-2. Arrancar con Compose y comprobar el estado 3
3. Cambiar PERIODO a 5 en el .env 4
4. Tercer sensor y servicio web con nginx 4
5. Parar el broker 6
6. docker compose config 6
7. docker compose down 7
Comprueba que funciona 7
Tabla de decisiones 7
Estado final 8

Nota: desde el reto 2 de D2, el simulador publica también la humedad, por eso aparece en los registros. La imagen la construye Compose a partir de `../02-simulador` con la etiqueta `aula/simulador:1.0`.

1-2. Arrancar con Compose y comprobar el estado

```
Copy-Item .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs --tail 3
```

Compose construye la imagen (todas las capas salen de la caché de D2), crea la red `demo-sensores_default`, arranca el broker y **espera a que esté healthy** antes de arrancar los sensores (`depends_on` con `condition: service_healthy`). En `docker compose ps` el broker aparece como `(healthy)` y los sensores llevan menos tiempo encendidos que él. En lugar de `logs -f`, que no termina nunca, se usa `logs --tail 3` para la captura.

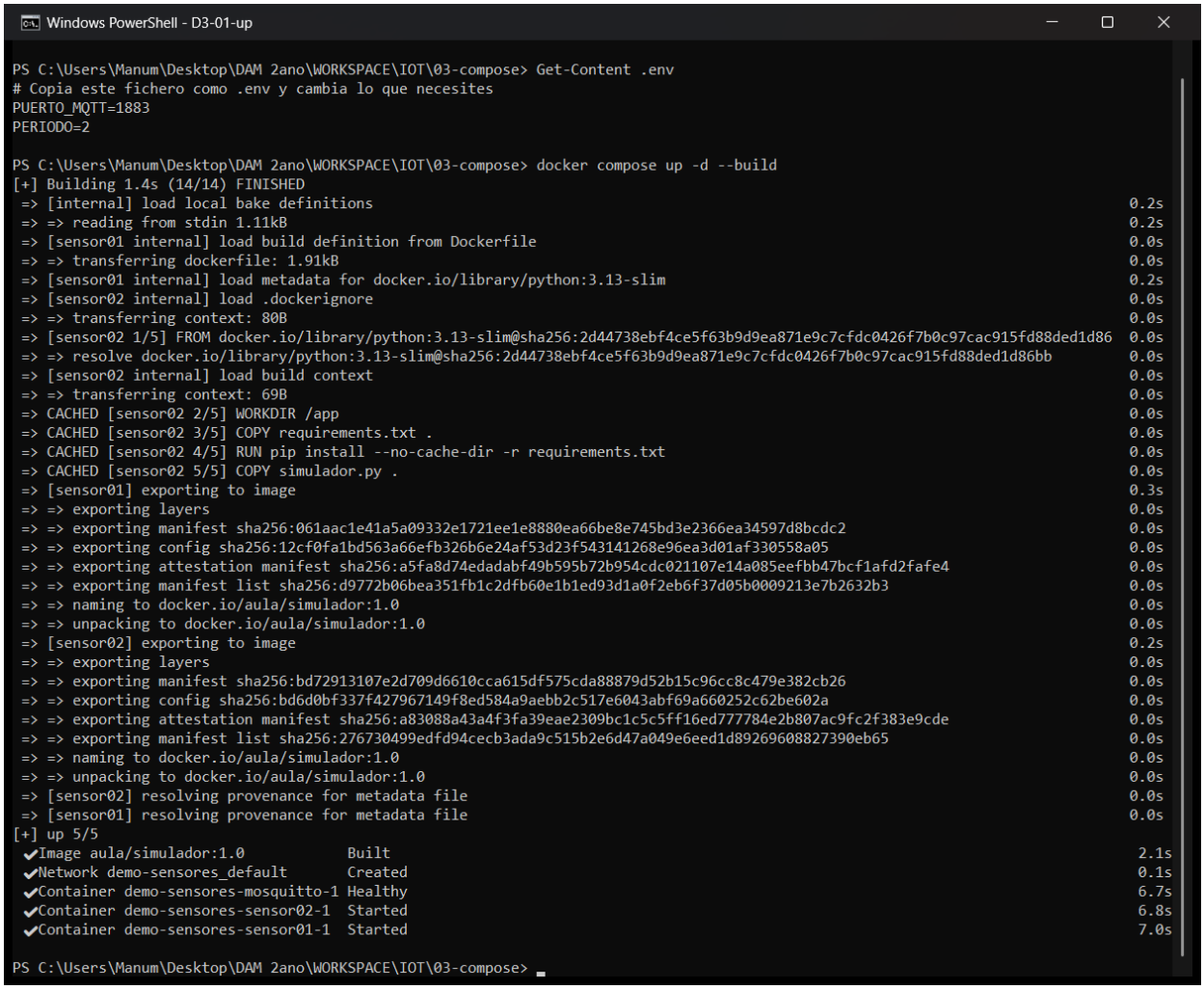


Figura 1. *docker compose up -d --build: el broker queda Healthy y después arrancan los sensores.*

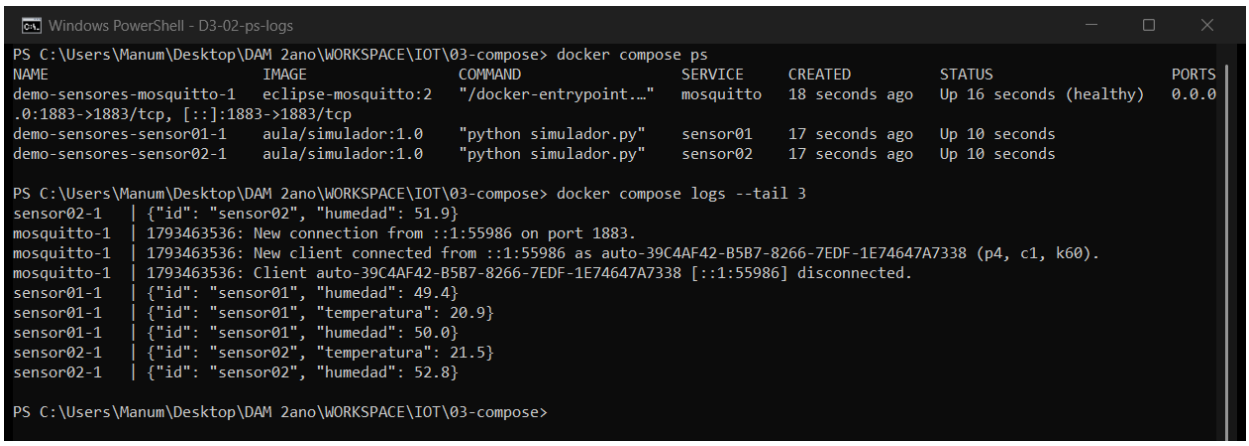


Figura 2. *Estado de los servicios y últimos mensajes de cada uno.*

3. Cambiar PERIODO a 5 en el .env

```
(Get-Content .env) -replace "PERIODO=2","PERIODO=5" | Set-Content .env
docker compose up -d
```

Contenedores recreados: solo `sensor01` y `sensor02`, que son los que usan `PERIODO`. El broker no cambia. Para confirmarlo se compararon los identificadores de los contenedores antes y después:

```
ANTES:  mosquitto 6324bbdf555c   sensor01 1e8d0cf559c1   sensor02 c2fdbf2bfa59
 Container demo-sensores-mosquitto-1 Running
 Container demo-sensores-sensor01-1 Recreate
 Container demo-sensores-sensor02-1 Recreate
DESPUÉS: mosquitto 6324bbdf555c  sensor01 6902d90fffdf   sensor02 84b5b88cf038
```

Compose compara la configuración final de cada servicio con la del contenedor que está en marcha y solo recrea los que difieren.

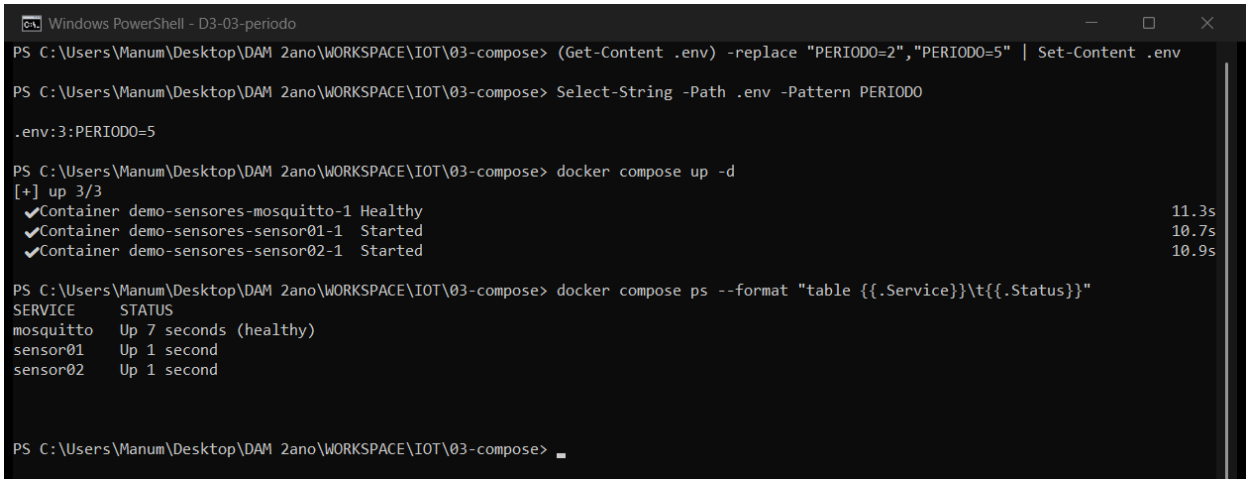


Figura 3. *Tras cambiar el .env, docker compose up -d solo recrea los sensores.*

4. Tercer sensor y servicio web con nginx

Se añadieron dos servicios al `compose.yaml`: `sensor03`, con la misma imagen y su propio `SENSOR_ID`, y `web`, con `nginx:alpine` sirviendo la carpeta de `01-web` en el puerto 8080 (montada en solo lectura):

```
  sensor03:
    image: aula/simulador:1.0
    environment:
      SENSOR_ID: sensor03
      PERIODO: ${PERIODO:-2}
    depends_on:
      mosquitto:
        condition: service_healthy
    restart: unless-stopped
```

```
  web:
    image: nginx:alpine
    ports:
      - "${PUERTO_WEB:-8080}:80"
    volumes:
      - ../01-web/01-web/html:/usr/share/nginx/html:ro
    restart: unless-stopped
```

Al hacer `docker compose up -d` solo se crean los dos servicios nuevos; los que ya estaban siguen `Running`.

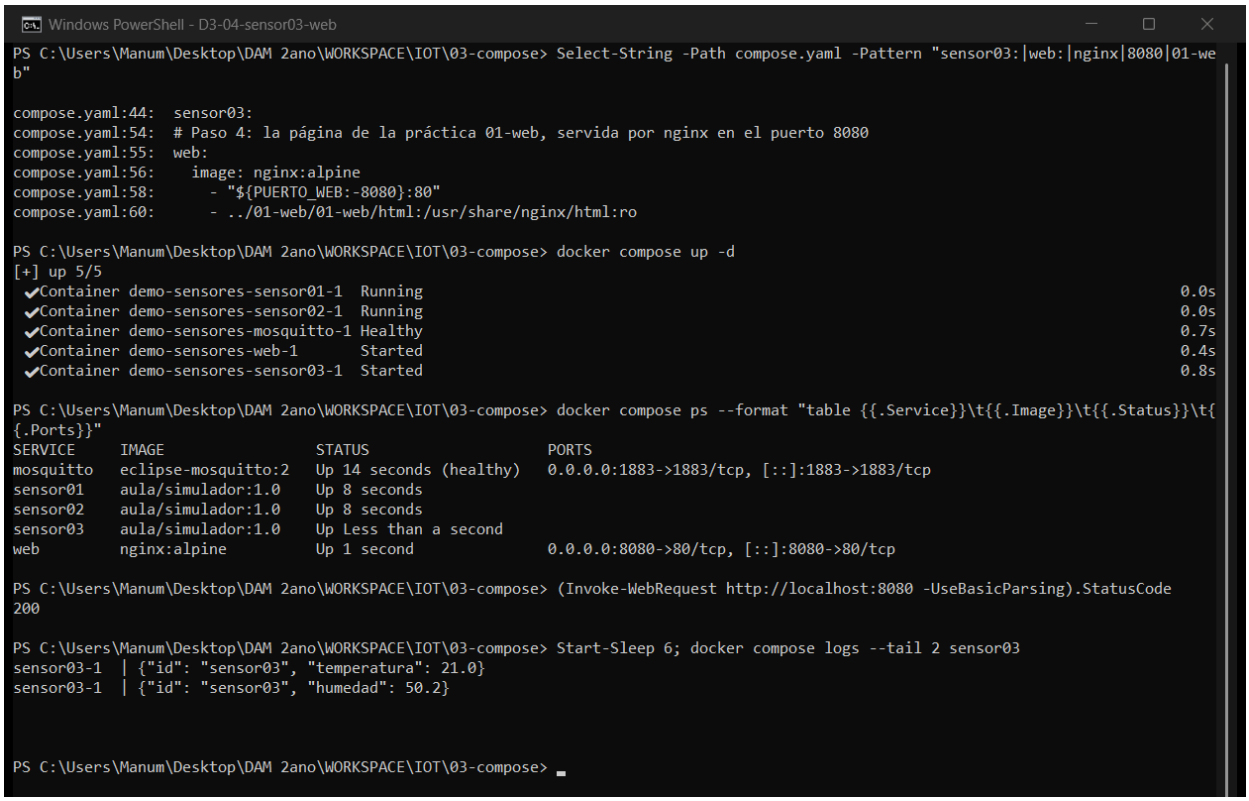


Figura 4. *Los cinco servicios en marcha; la web responde con código 200.*

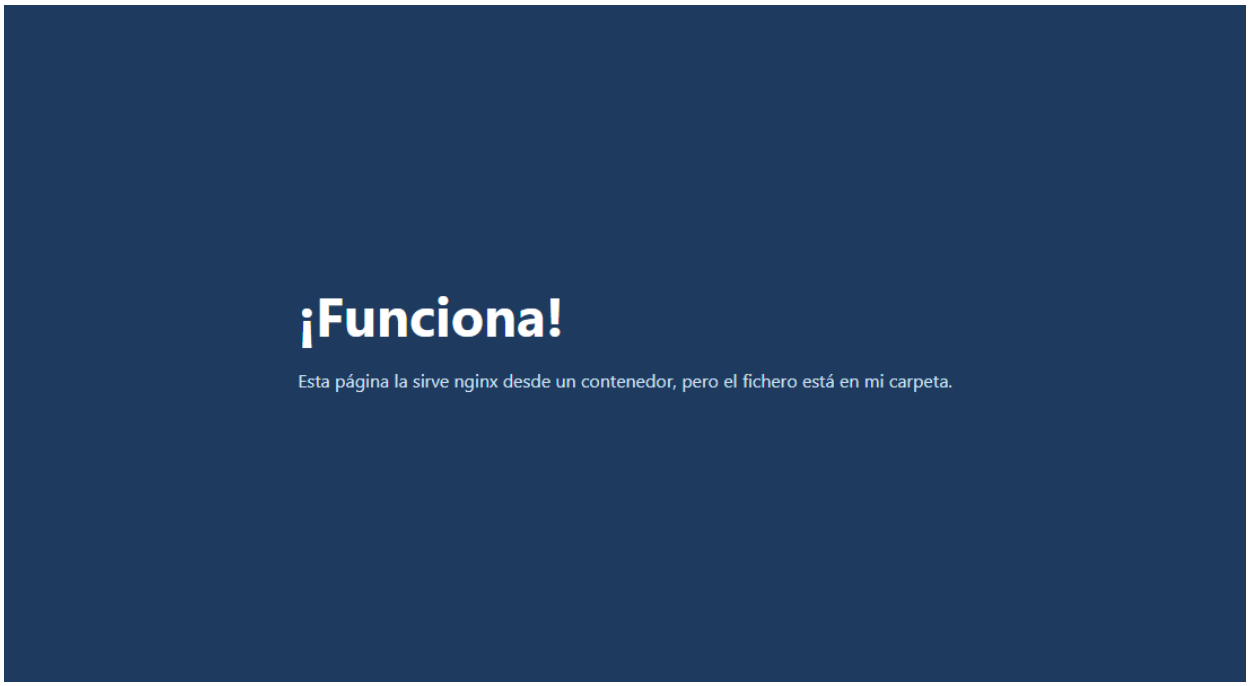


Figura 5. *La página de 01-web servida por el servicio web en http://localhost:8080.*

5. Parar el broker

```
docker compose stop mosquitto
docker compose start mosquitto
```

Qué les pasa a los sensores: no se caen ni se reinician (0 reinicios según `docker inspect`). El programa sigue en marcha porque la librería paho-mqtt, con `loop_start()`, intenta reconectar sola en segundo plano. Mientras el broker está parado, los mensajes no llegan a ninguna parte. Cuando el broker vuelve, los sensores se reconectan solos y los tres vuelven a publicar. Se comprobó con un suscriptor:

```
/demo-sensores-sensor01-1 restarts=0
/demo-sensores-sensor03-1 restarts=0
demo/sensor03/temperatura {"id": "sensor03", "temperatura": 21.5}
demo/sensor02/temperatura {"id": "sensor02", "temperatura": 21.6}
demo/sensor01/temperatura {"id": "sensor01", "temperatura": 22.2}
```

La política `restart: unless-stopped` actúa cuando el **proceso termina con error**, por ejemplo si un sensor arranca sin broker (en D2 se vio que sale con código 1). En ese caso Docker lo vuelve a arrancar hasta que el broker esté disponible.

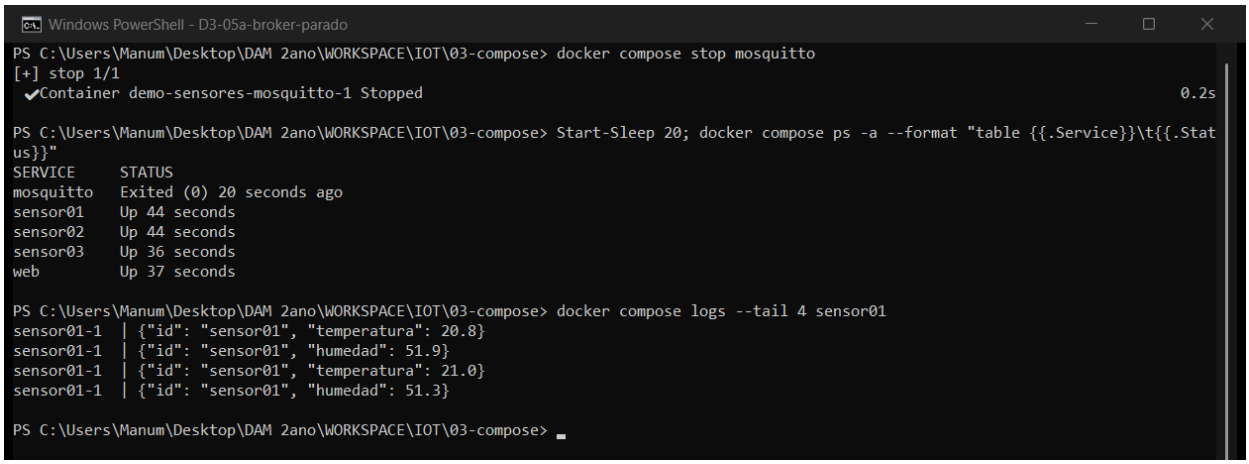


Figura 6. *Con el broker parado, los sensores siguen en marcha.*

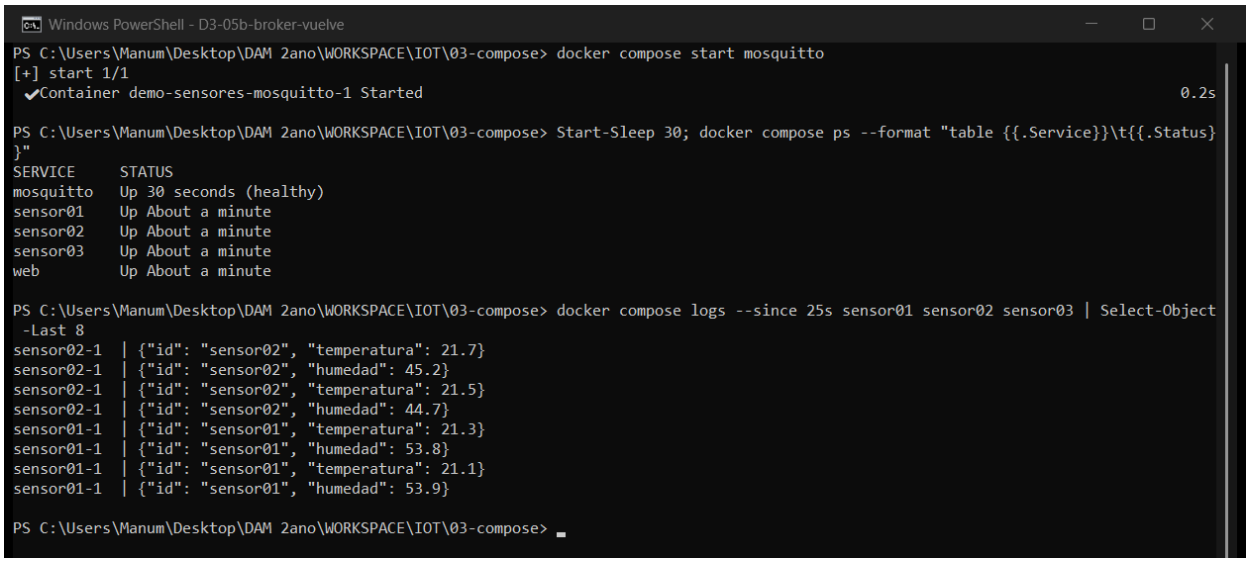


Figura 7. *El broker vuelve (healthy) y los sensores siguen publicando.*

6. docker compose config

Muestra el compose final, con todas las variables sustituidas. `${PERIODO:-2}` aparece como `"5"` (valor del `.env`), `${PUERTO_MQTT:-1883}` como `published: "1883"` y la ruta relativa de la web, convertida en ruta absoluta.

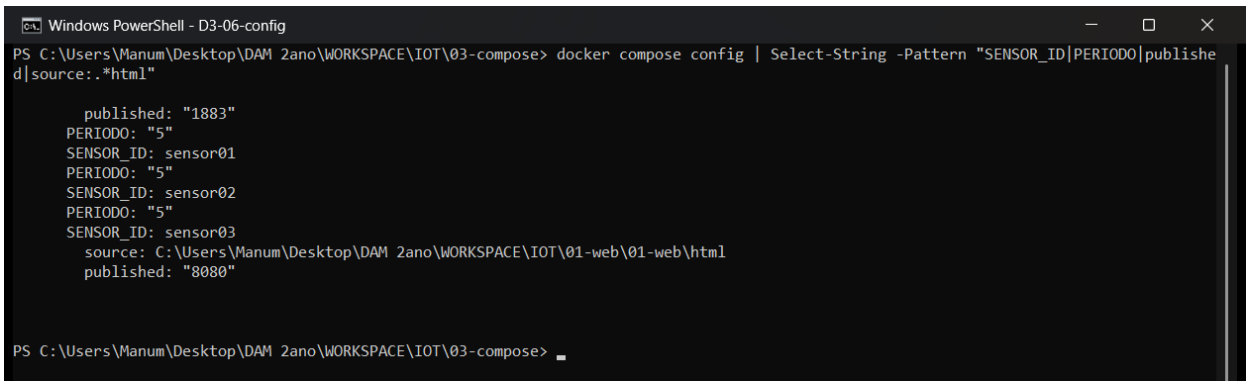


Figura 8. *Variables ya sustituidas en la salida de docker compose config.*

7. docker compose down

Para y elimina los cinco contenedores y la red con un solo comando.



Figura 9. *docker compose down elimina contenedores y red.*

Comprueba que funciona

- **Broker healthy y sensores después:** sí (Figuras 1 y 2).
- **Tras cambiar el .env, solo se recrean los servicios afectados:** sí, solo los sensores (paso 3).
- **Con el broker parado, los sensores se recuperan solos:** sí, aunque no gracias a la política `restart`, sino a la reconexión automática de paho-mqtt (paso 5).

Tabla de decisiones

Decisión	mosquitto	sensor01-03	web
Imagen y versión	`eclipse-mosquitto:2` (oficial)	`aula/simulador:1.0` (propia, D2)	`nginx:alpine` (oficial, ligera)
Puertos publicados	1883, para que clientes de fuera puedan publicar o suscribirse	Ninguno: solo publican hacia el broker	8080 → 80, para verla en el navegador
Datos persistentes	No hacen falta en las pruebas	No: no guardan estado	La página se monta desde `01-web` (`:ro`)
Configuración	`mosquitto.conf` montado en solo lectura; puerto en `.env`	`SENSOR_ID` y `PERIODO` por variables	Puerto en `.env` (`PUERTO_WEB`)
Dependencias	Ninguna	`mosquitto` con `service_healthy`	Ninguna
Comprobación de salud	`mosquitto_sub` a `$SYS/broker/uptime`	No tienen	No tiene

Decisión	mosquitto	sensor01-03	web
Política de reinicio	`unless-stopped`	`unless-stopped`	`unless-stopped`

Estado final

`03-compose/compose.yaml` incluye `sensor03` y el servicio `web`, y el `.env` queda con `PERIODO=2`. Al terminar no queda ningún contenedor en marcha.